Université Paris Cité

Neural Networks from scratch

24 June, 2025

Lyu Chen, Shuyue GU, Varvara SMIRNOVA
Supervised by Marie Candito

Presentation Overview

1. Introduction & Task Definition
2. Theoretical Background & Core Architecture
3. Code Structure & Main Components
4. Preliminary Experiments
5. POS Tagging Task
6. Results
7. Future Work

Introduction & Task definition

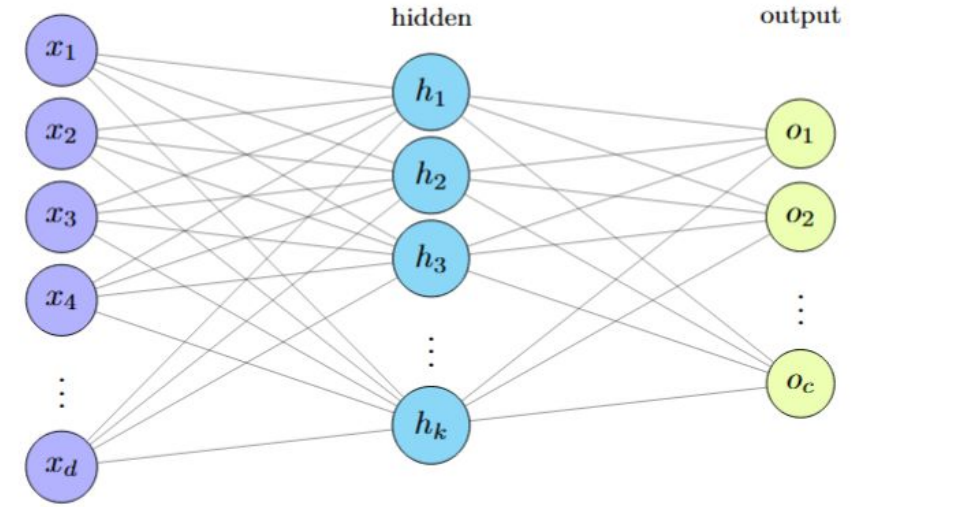
Goal: Build and evaluate a feedforward neural network from scratch for POS tagging task

Motivation: understanding how deep learning works at the lowest level

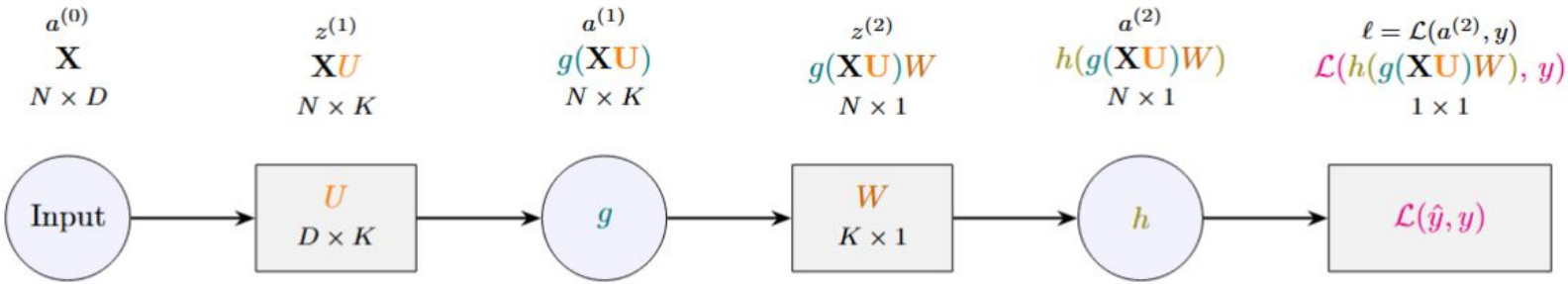
Tasks performed: binary classification (XOR), multiclass, POS tagging

Introduction & Task definition

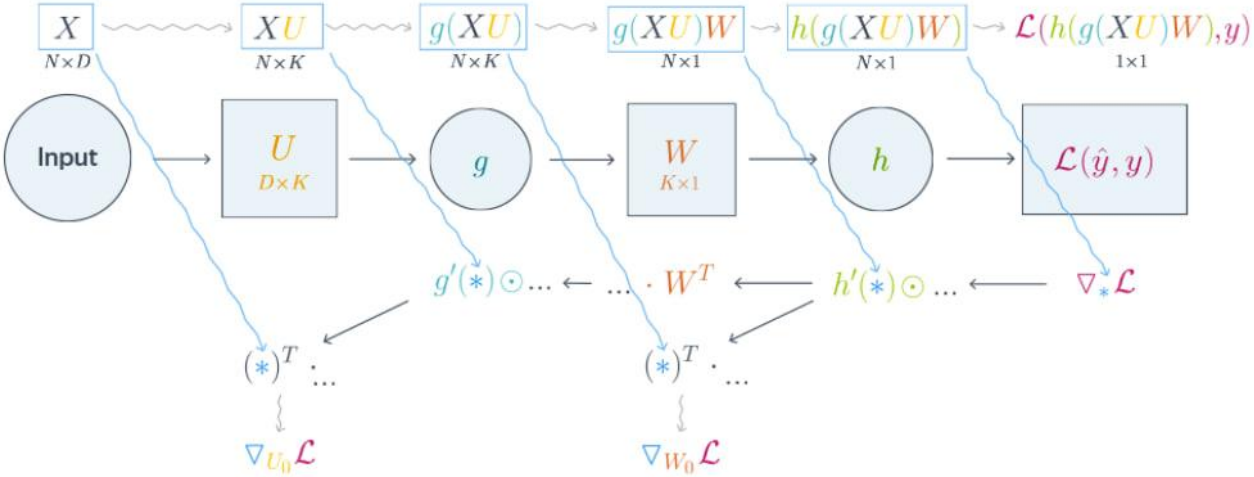
Definition: A feedforward neural network is a parametric, fully differentiable function, typically represented as a composition of several linear and nonlinear transformations applied sequentially to an input vector X



Theoretical background: Forward pass



Theoretical background: Backward pass



Starting from the loss, the gradient flows backward through each component of the network:

Input weights ***U*** ← Nonlinearity ***g*** ← Linear weights ***W*** ← Output activation

$$\nabla_{z^{(1)}}\mathcal{L} = \left(\nabla_{z^{(2)}}\mathcal{L}\cdot\mathbf{W}^\top\right)\circ g'\left(z^{(1)}\right) \qquad \nabla_{z^{(2)}}\mathcal{L} = \nabla_{a^{(2)}}\mathcal{L}\circ h'\left(z^{(2)}\right)$$

$$\nabla_U\mathcal{L} = X^\top\cdot\nabla_{z^{(1)}}\mathcal{L} \qquad \nabla_W\mathcal{L} = (g(XU))^\top\cdot\nabla_{z^{(2)}}\mathcal{L}$$

Theoretical background

Cross-Entropy Loss

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log(\hat{y}_{ic})$$

Intuition: high penalty if model assigns low probability to the correct class

Regularization

L2 regularization: $\Omega(\theta) = \lambda \sum_{k} \|W^{(k)}\|_F^2$

Adds penalty term to loss: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cross-entropy}} + \lambda \cdot \Omega(\theta)$

Optimization: Stochastic Gradient Descent

Updates weights to reduce the loss: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}$

Uses mini-batches

Includes learning rate η, optionally supports early stopping

Core Architecture: class LinearLayer

Affine Transformation: Input → Output

Key responsibilities:

Forward pass: computes output for the next layer

Backward pass: computes gradients for weights, biases and input

Stores Internally:

Trainable parameters: $W \in R^{d*n}$, $b \in R^d$

Cached input x for backpropagation

Role in Network

Transforms input to a learned representation

Used in each layer before applying non-linearity

Core Architecture: class ActivationFunction

Introducing Non-Linearity

→ Without non-linearity, the network is just a linear transformation,

regardless of depth

What it does:

Takes in preactivation ***a^(k)***

Applies a non-linear function ***h^(k) = g(a^(k))***

Stores ***a^(k)*** for the backward pass

Supported types: ReLU, Sigmoid, Tanh, Softmax

Function	Output Range	Derivative Expression
ReLU	$[0, +\infty)$	$\frac{d}{dx} = 1$ if $x > 0$, else 0
Sigmoid	$(0, 1)$	$\sigma(x)(1 - \sigma(x))$
Tanh	$(-1, 1)$	$1 - \tanh^2(x)$
Softmax	$(0, 1), \sum p_i = 1$	$\frac{\partial p_i}{\partial z_j} = \begin{cases} p_i(1 - p_i) & \text{if } i = j \\ -p_i p_j & \text{if } i \neq j \end{cases}$

Backward Pass: computes the gradient of the loss with respect to the pre-activation

$$\nabla_{a^{(k)}} \mathcal{L} = \nabla_{h^{(k)}} \mathcal{L} \odot g'(a^{(k)})$$

Core Architecture: `class NeuralNetwork`

Central controller that chains layers together, computes the loss, and runs backpropagation.

Key responsibilities:

Orchestrates forward pass: sequentially applies LinearLayer → Activation → …, returns \hat{y}

Computes cross-entropy loss over batches

Performs backpropagation: starts from loss derivative, iterates backward through layers, applies gradients to update parameters

Configurable Architecture

Takes list of layer configs: [(output_1, act_1, input_1), (output_2, act_2, input_2), ...]

Makes the model **modular** and easily adaptable

Core Architecture: `class Embedding Layer`

Central module that maps sparse one-hot vectors into dense embedding space and optionally fine-tunes embeddings.

Key responsibilities:

Embedding Lookup: Converts sparse one-hot encoded inputs to dense vector representations.

Supports Fine-tuning: Optionally updates embedding matrix through backpropagation.

Handles Input Windowing: Processes concatenated embeddings from context windows.

Configurable Architecture:

- Takes pretrained or randomly initialized embeddings as input.
- Adjustable learning rate for embeddings.
- Embeddings can be set to trainable or fixed for different tasks.

Preliminary Experiments

- The classical benchmark XOR

- The randomly simple classification task

The classical benchmark XOR

What is XOR?

- Classic non-linear binary classification task.
- Fundamental test for neural networks' ability to learn non-linear patterns.

x1	x2	y
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

XOR Truth Table

How do we know it really works?

We verify neural network effectiveness through:

- **Loss Trend**: Loss should consistently decrease and stabilize over epochs.
- **Accuracy**: Predicted classifications should closely match the true classifications.
- **Detailed Predictions**: Outputs should clearly indicate correct class probabilities.
- **Network Capacity Comparison**: Networks with larger hidden layers should outperform smaller networks significantly, demonstrating capacity to handle non-linear tasks.

XOR Classification Experiment

Experiment Setup:
Tested networks with hidden layers of:

- 100 neurons
- 50 neurons
- 2 neurons

Training Details:

- Epochs: 5000
- Learning rate: 0.1
- Loss function: Cross-entropy

Loss trend and accuracy of 2 neurons'



```
Epoch 0: Loss = 0.6932, Accuracy = 0.50
Epoch 500: Loss = 0.4596, Accuracy = 0.75
Epoch 1000: Loss = 0.3539, Accuracy = 0.75
Epoch 1500: Loss = 0.3494, Accuracy = 0.75
Epoch 2000: Loss = 0.3484, Accuracy = 0.75
Epoch 2500: Loss = 0.3477, Accuracy = 0.75
Epoch 3000: Loss = 0.3475, Accuracy = 0.75
Epoch 3500: Loss = 0.3473, Accuracy = 0.75
Epoch 4000: Loss = 0.3471, Accuracy = 0.75
Epoch 4500: Loss = 0.3471, Accuracy = 0.75
```

XOR Classification Experiment

Loss trend and accuracy of 100 neurons’

Epoch 0: Loss = 0.6932, Accuracy = 0.25

Epoch 500: Loss = 0.0823, Accuracy = 1.00

Epoch 1000: Loss = 0.0172, Accuracy = 1.00

Epoch 1500: Loss = 0.0089, Accuracy = 1.00

Epoch 2000: Loss = 0.0059, Accuracy = 1.00

Epoch 2500: Loss = 0.0043, Accuracy = 1.00

Epoch 3000: Loss = 0.0034, Accuracy = 1.00

Epoch 3500: Loss = 0.0028, Accuracy = 1.00

Epoch 4000: Loss = 0.0023, Accuracy = 1.00

Epoch 4500: Loss = 0.0020, Accuracy = 1.00

Loss trend and accuracy of 50 neurons’

Epoch 0: Loss = 0.6932, Accuracy = 0.25

Epoch 500: Loss = 0.1039, Accuracy = 1.00

Epoch 1000: Loss = 0.0190, Accuracy = 1.00

Epoch 1500: Loss = 0.0096, Accuracy = 1.00

Epoch 2000: Loss = 0.0063, Accuracy = 1.00

Epoch 2500: Loss = 0.0046, Accuracy = 1.00

Epoch 3000: Loss = 0.0037, Accuracy = 1.00

Epoch 3500: Loss = 0.0030, Accuracy = 1.00

Epoch 4000: Loss = 0.0026, Accuracy = 1.00

Epoch 4500: Loss = 0.0022, Accuracy = 1.00

XOR Classification Experiment

Final predictions' probabilities for the three models(respectively 100, 50, 2)

Input: [0 0] → Predicted: [0.99522864 0.00477136] (label: 0), True: 0
Input: [0 1] → Predicted: [8.14107282e-04 9.99185893e-01] (label: 1), True: 1
Input: [1 0] → Predicted: [8.09680597e-04 9.99190319e-01] (label: 1), True: 1
Input: [1 1] → Predicted: [9.99423522e-01 5.76477927e-04] (label: 0), True: 0
Final Accuracy: 4/4

Input: [0 0] → Predicted: [0.9943179 0.0056821] (label: 0), True: 0
Input: [0 1] → Predicted: [8.00274859e-04 9.99199725e-01] (label: 1), True: 1
Input: [1 0] → Predicted: [7.99752276e-04 9.99200248e-01] (label: 1), True: 1
Input: [1 1] → Predicted: [9.99506450e-01 4.93550361e-04] (label: 0), True: 0
Final Accuracy: 4/4

Input: [0 0] → Predicted: [0.50021151 0.49978849] (label: 0), True: 0
Input: [0 1] → Predicted: [0.50021151 0.49978849] (label: 0), True: 1
Input: [1 0] → Predicted: [7.15649015e-04 9.99284351e-01] (label: 1), True: 1
Input: [1 1] → Predicted: [0.9988876 0.0011124] (label: 0), True: 0
Final Accuracy: 3/4

Multiclass Classification Test

Task Description: 3-class classification with input dimension of 10.

Neural Network Architecture:

- Input layer (10-dim)
- Hidden layer (16 neurons, ReLU activation)
- Output layer (3 neurons, Softmax activation)

Training parameters: 1000 epochs, learning rate = 0.1

Result: Accurate classification demonstrating network's ability for multiclass tasks.

POS Tagging Task with One-hot Representation

Corpus: Universal Dependencies English ParTUT (en_partut-ud) corpus.

Data Splits: Train, validation, and test sets based on standard .conllu files.

Data Preparation:

- All words lowercased.
- Special tokens <PAD> and <UNK> added for padding and unknown words.
- Vocabulary built from training data; tags from POS labels.
- Each word converted to a one-hot vector with vocabulary size dimension.
- Tags also one-hot encoded using POS tag set.

POS Tagging Task with One-hot Representation

Windowing:

- Inputs are concatenations of one-hot vectors from a context window (e.g., previous, current, next word).
- Outputs are POS tags for the current word.

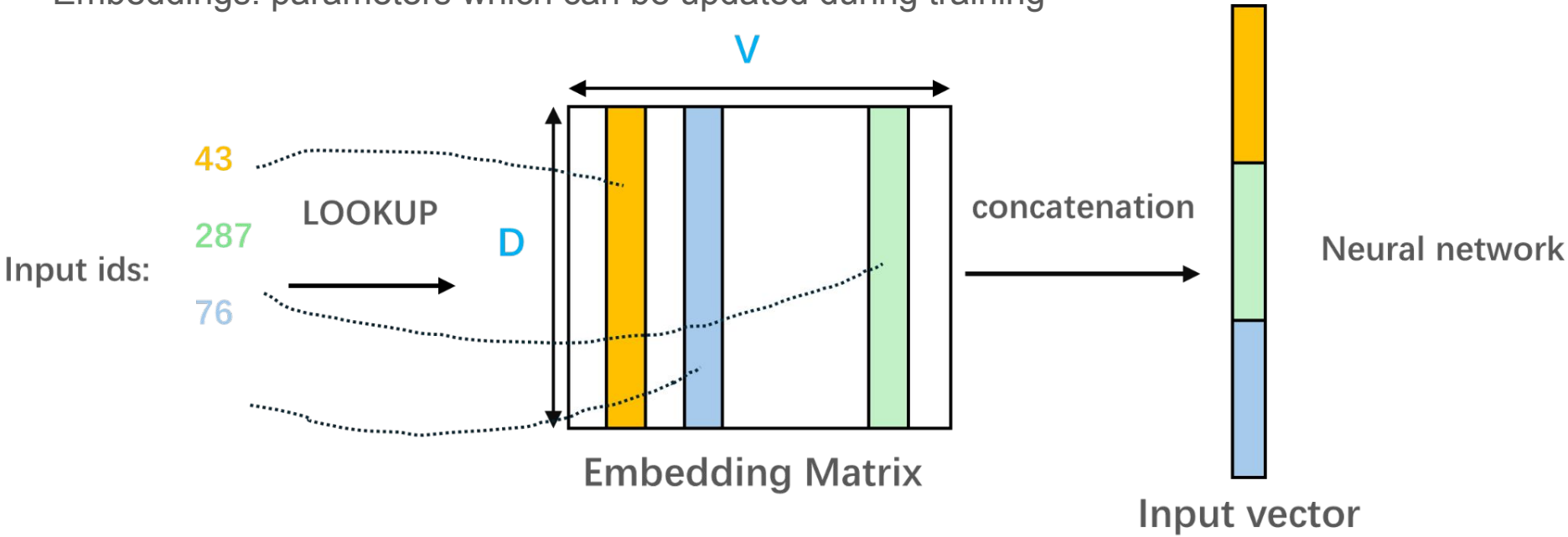
Neural Network Structure:

- **Input dimension:** Vocabulary size × (window size × 2 + 1)
- **Hidden layer:** 128 neurons with ReLU activation
- **Output layer:** Number of POS tags with Softmax activation
- **Loss function:** Cross-entropy loss

POS Tagging Task with pre-trained embeddings Representation

Input: concatenations of **GloVe embeddings** from previous, current and next word.

Embeddings: parameters which can be updated during training



POS Tagging Task results

Metric	Embeddings Model	One-hot Model
Overall Accuracy	83.9%	89.8%
Macro-average F1	73.3%	79.8%
Weighted F1	84.2%	89.9%

POS Tagging Task results

One-hot model

	precision	recall	f1-score	support
ADJ	0.719	0.814	0.763	226
ADP	0.938	0.988	0.962	486
ADV	0.935	0.672	0.782	128
AUX	0.966	0.979	0.972	234
CCONJ	1.000	0.969	0.984	96
DET	0.980	0.991	0.985	439
INTJ	0.000	0.000	0.000	2
NOUN	0.924	0.874	0.898	754
PART	0.909	0.896	0.902	67
PRON	0.968	0.844	0.902	109
PROPN	0.507	0.800	0.621	90
SCONJ	0.825	0.673	0.742	49
VERB	0.870	0.844	0.857	326

Embeddings model

	precision	recall	f1-score	support
ADJ	0.5992	0.6814	0.6377	226
ADP	0.9505	0.9486	0.9495	486
ADV	0.8395	0.5312	0.6507	128
AUX	0.9777	0.9359	0.9563	234
CCONJ	1.0000	0.9792	0.9895	96
DET	0.9749	0.9727	0.9738	439
INTJ	0.0000	0.0000	0.0000	2
NOUN	0.8725	0.7984	0.8338	754
PART	0.5686	0.8657	0.6864	67
PRON	0.8016	0.9266	0.8596	109
PROPN	0.3862	0.6222	0.4766	90
SCONJ	0.8293	0.6939	0.7556	49
VERB	0.7647	0.7577	0.7612	326

Result Analysis

One-hot model outperforms the embeddings model across most metrics and POS categories.

Using other embeddings like FastText and more complex nn structure does not improve the embedding model performance

- Limited training data
- Pretrained embeddings are not task-specific
- Loss of Precision
- Embedding Layer Bottleneck: loss of critical information when using shallow model

Future Work

Use **contextualized embeddings**: word embeddings generated by BERT depending on its surrounding context

Try **Sequential Architectures** like Transformers

Hybrid Representations: **combine one-hot and embeddings** to benefit from both lexical precision and semantic generalization

Optimization Improvements: AdamW optimizer, dropout, regularization

Use **more training data** with **balanced label distribution**

References

Marie Candito. Machine learning for nlp 1 – course notes, 2024. M1 Computational Linguistics, Université Paris Cité. Last modified: September 17, 2024.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL `https://www.deeplearningbook.org/`.

Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Pearson, 4th edition, 2025. URL `https://web.stanford.edu/~jurafsky/slp3/`. Draft of January 12, 2025.

Hugo Larochelle. Deep learning: Neural networks – lecture slides, 2020. URL `https://www.youtube.com/watch?v=p5tL2JqCRDo&list=PL6Xpj9I5qXYEcOhn7TqghAJ6NAPrNmUBH&index=12&ab_channel=HugoLarochelle`. Université de Montréal, MILA.

Merci